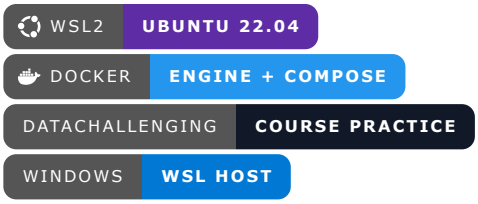
《大数据分析技术》DataChallenging 学生手册

API 调用 · WSL2 · Ubuntu 22.04 LTS · Docker Engine · Docker Compose · DataChallenging



📚 手册导航目录

《大数据分析技术》DataChallenging 环境安装与实践手册

> **使用说明**：点击目录中的章节名称，可快速跳转至对应内容。首次安装环境的同学建议按顺序阅读；已经完成环境安装的同学，可直接进入 Phase 1 或 Phase 2。

第一部分：课程概览与学习指南

- 1.1 文档速览：这份手册可以帮助你完成什么
- 1.2 推荐阅读顺序
- 1.3 Phase 1：设计 Task 和 Agent
- 1.4 Phase 2：匿名题目分析与方案优化
- 1.5 DataChallenging 推荐实践流程

第二部分：环境安装与配置

2.1 大模型 API 配置

- 获取阿里云百炼 API Key
- 查找 WorkspaceId 和配置 BaseURL

2.2 WSL2 与 Ubuntu 环境安装

- 安装前提与系统要求
- 打开 Windows PowerShell 管理员模式
- 更新 WSL
- 安装 Ubuntu 22.04 LTS

- 首次进入 Ubuntu
- 使用 root 用户进入 Ubuntu
- 启用 systemd
- 检查 WSL 与 Ubuntu 安装结果

2.3 Docker Engine 与 Docker Compose 安装

- 方案一：通过 Docker 官方软件源安装
- 方案二：通过 Ubuntu 仓库安装
- 检查 Docker Engine 与 Compose 是否安装成功

2.4 启动 DataChallenging 本地环境

- 启动 Docker 服务
- 进入课程部署目录
- 首次加载镜像并启动课程容器
- 日常快速启动
- 检查 DataChallenging CLI

第三部分：Phase 1 — Task 与 Agent 初始设计

3.1 认识课程项目文件

- 创建 Task 工作目录
- 理解 task.json、data.csv、reference.py、agent.py 的作用

3.2 设计数据分析 Task

- 编写 Task：题目要求与设计原则
- 准备 CSV 数据文件
- 了解 Failure Mode（14 类错误类型）

3.3 编写标准答案与数据分析 Agent

- 编写 reference.py：标准答案程序
- 编写 agent.py：数据分析 Agent
- 理解 reference.py 与 agent.py 的区别

3.4 本地测试与提交 Phase 1

- 运行 Task 与 Agent
- 构建 Phase 1 提交包
- 验证最终提交 ZIP

3.5 理解 Agent 运行结果

- 情况一：Agent 正确完成任务
- 情况二：Agent 运行成功但答案错误
- 情况三：Agent 运行失败

第四部分：Phase 2 — 匿名任务挑战与优化

4.1 匿名 Task 分析

- 了解匿名 Task 分析任务
- 导入匿名任务
- 使用 Agent v1 测试 10 个匿名 Task
- 分析题目歧义、数据问题和 Agent 失败原因

4.2 优化 Task 与 Agent

- 修改并确定 Task v2
- 修改并确定 Agent v2
- 运行并测试最终 Task-Agent

4.3 构建与验证 Phase 2 提交包

- 构建 Phase 2 提交 ZIP
- 验证最终 Phase 2 提交包

4.4 匿名任务评审

- 填写 review-comments.json
- 匿名 Task 评分与评语要求

4.5 实验报告编写

- 实验报告总体要求
- Task v1 与 Task v2 的设计和迭代

- Agent v1 与 Agent v2 的设计和迭代
- 实验总结与反思
- 最终实验报告结构

第五部分：常见问题与提交前检查

5.1 常见问题排查

- 常见问题速查：Docker、CLI、运行及答案错误

5.2 提交前检查

- 每次提交前自检

5.3 快速操作参考

- 最小操作版：常用命令与运行流程
- DataChallenging 一句话理解

阅读建议

- **第一次使用**：第一部分 → 第二部分 → 第三部分。
- **已经安装好环境**：直接进入第三部分，开始 Phase 1。
- **收到匿名任务包**：直接进入第四部分，完成 Phase 2。
- **遇到运行错误或提交失败**：优先查看第五部分。

📌 文档速览

模块	你要完成什么	成功标志
🧭 API 调用	登录阿里云百炼，查看额度，创建 API Key，确认 BaseURL	拿到模型名、额度、BaseURL
🐧 WSL Ubuntu	在 Windows 上安装 WSL2 + Ubuntu 22.04 LTS	`wsl -l -v` 显示 `VERSION 2`
👑 root + systemd	以 root 进入 Ubuntu，并启用 systemd	`whoami=root`，`ps -p 1 -o comm=` 输出 `systemd`
🐳 Docker Engine	在 WSL Ubuntu 内安装 Docker，不走 Docker Desktop	`docker version` 显示 Client / Server
🧩 Docker Compose	安装并验证 Compose	`docker compose version` 正常输出

模块	你要完成什么	成功标志
DataChallenging	启动课程实践环境，创建 Task 与 Agent，并完成本地测试	`datac --help` 正常，Task/Agent 可运行

推荐阅读顺序

1. **API 调用**：先确认阿里云百炼登录、额度、API Key 和 BaseURL。
2. **WSL Ubuntu**：安装 Windows + WSL2 + Ubuntu 22.04 LTS。
3. **root + systemd**：设置 DataChallenging 所需的 Docker 运行环境。
4. **Docker Engine & Compose**：选择 Docker 官方源或 Ubuntu 仓库路线。
5. **DataChallenging 本地环境**：启动课程提供的本地服务，并确认 CLI 与浏览器入口可用。
6. **课程实践**：按 Task → Data → reference.py → Agent → Run → Build 的顺序完成阶段一，并进入匿名题目分析与迭代。

API 配置

获取 API Key

1. 登录 阿里云大学
2. 进入 百炼控制台 获取免费配额
3. 在 API Key 管理页面创建密钥

业务空间 ID决定了你的BaseURL格式

你已经创建的是 **华北 2（北京）的百炼按量计费 API Key**，那么你的 OpenAI 兼容 BaseURL 格式就是：

```
https://{WorkspaceId}.cn-beijing.maas.aliyuncs.com/compatible-mode/v1
```

把 `{WorkspaceId}` 替换成你的真实 **业务空间 ID** 即可。阿里云官方文档明确说明，华北 2（北京）的按量计费 Base URL 是这个格式，且需要用真实业务空间 ID 替换 `WorkspaceId`。

如何找到你的 WorkspaceId

进入 **阿里云百炼控制台**，确认右上角地域是 **华北 2（北京）**，然后进入 **业务空间管理**，在列表里的 **Workspace ID** 列复制你 API Key 所属业务空间的 ID。官方文档说明：登录百炼控制台后，进入业务空间管理界面，在 `Workspace ID` 列查看并复制对应业务空间 ID。

也就是说，假设你的 Workspace ID 是：llm-abc123

那么你的 BaseURL 就是：https://llm-abc123.cn-beijing.maas.aliyuncs.com/compatible-mode/v1

BaseURL ： https://{WorkspaceId}.cn-beijing.maas.aliyuncs.com/compatible-mode/v1

模型	当前剩余额度
`qwen3.6-flash`	**999,958 / 1,000,000**
`qwen3.7-plus`	1,000,000 / 1,000,000
`deepseek-v4-flash`	1,000,000 / 1,000,000

注意：这是 **免费额度**，不是账户现金余额。阿里云官方说明，百炼会给各模型发放新人免费额度，免费额度仅用于抵扣模型实时推理调用；额度到期或耗尽后，继续调用会产生计费。(阿里云帮助中心)

系统抵扣顺序一般是 **免费额度优先**，免费额度用完后才会开始抵扣资源包或产生按量费用。(阿里云帮助中心)

🐧 ——— Linux / WSL ——— 🐧

Windows 安装 WSL2 → 在线安装 Ubuntu → 在 Ubuntu / WSL 内启用 systemd → 在线安装 Docker Engine + Docker Compose 插件 → 导入离线镜像 → 解压项目包 → 使用 Docker Compose 启动容器 → Windows 浏览器访问本地端口。

本手册采用"在 WSL Ubuntu 内安装 Docker Engine"的方式，不依赖 Docker Desktop。Microsoft 官方文档说明，`wsl --install` 可一键安装 WSL 并默认安装 Ubuntu，新安装的 Linux 发行版默认使用 WSL 2；Docker 官方文档推荐在 Ubuntu 中通过 Docker 的 apt 仓库安装 Docker Engine 与 `docker-compose-plugin` 。

🐧 ——— WSL Ubuntu ——— 🐧

下面是**只安装 WSL2 + Ubuntu 22.04 LTS** 的完整流程。

这一步是在后续 Docker Engine / DataChallenging 课程实践之前完成的基础环境；

强调：wsl 命令一般在 Windows PowerShell 执行，

`apt` 、 `systemctl` 、 `docker` 等命令才是在 WSL Ubuntu 终端中执行 。

第一步. 前提确认

适用系统：**Windows 11 或 Windows 10 版本 2004 及以上，Build 19041 及以上**。Microsoft 官方 WSL 安装文档说明，下面的 `wsl --install` 命令要求 Windows 10 version 2004+ / Build 19041+ 或 Windows 11。(Microsoft Learn)

建议先确认虚拟化已开启：

1. 打开任务管理器。
2. 进入 **性能 → CPU**。
3. 看右下角 **虚拟化** 是否为 **已启用**。

Ubuntu 官方 WSL 文档也提醒：WSL 需要启用虚拟化，`wsl --install` 会尝试启用 Virtual Machine Platform，但通常需要重启后生效。(Ubuntu Documentation)

打开 Windows PowerShell 管理员模式

在 Windows 开始菜单搜索：

```
PowerShell
```

右键选择：

```
以管理员身份运行
```

后面的 `wsl ...` 命令都在这个 **Windows PowerShell 管理员窗口** 里执行。

更新 WSL

先执行：

```
wsl --update
```

再查看 WSL 版本：

```
wsl --version
```

Microsoft WSL 基础命令文档说明，`wsl --update` 用于更新 WSL，`wsl --version` 用于查看 WSL 及其组件版本。(Microsoft Learn)

查看可安装的 Ubuntu 版本

执行：

```
wsl --list --online
```

也可以写成：

```
wsl -l -o
```

你要找的是：

```
Ubuntu-22.04    Ubuntu 22.04 LTS
```

Ubuntu 官方 WSL 文档示例中，可安装列表包含 `Ubuntu-22.04 Ubuntu 22.04 LTS`，并说明可以用列表里的 `NAME` 安装指定版本。(Ubuntu Documentation)

用管理员 PowerShell 执行：

```
ipconfig /flushdns
netsh winsock reset
netsh winhttp reset proxy
```

执行完后重启电脑，再试：

```
wsl --list --online
```

安装 Ubuntu 22.04 LTS

执行：

```
wsl --install -d Ubuntu-22.04
```

如果你的 Windows / WSL 使用的是新版语法，也可能支持：

```
wsl --install Ubuntu-22.04
```

我建议用第一种，兼容性更好：

```
wsl --install -d Ubuntu-22.04
```

Microsoft 官方文档说明，`wsl --install -d <Distro>` 可以指定要安装的 Linux 发行版；如果没指定，默认会安装 Ubuntu。(Microsoft Learn)

如果下载卡在 0% 或 Microsoft Store 下载异常，试试：

```
wsl --install --web-download -d Ubuntu-22.04
```

Microsoft 文档也给出过这个处理方式：如果安装卡在 0.0%，可以加 `--web-download -d <DistroName>`。(Microsoft Learn)

重启 Windows

安装完成后，按提示重启电脑。

重启后，再打开 PowerShell，执行：

```
wsl -l -v
```

出现类似：

```
  NAME            STATE           VERSION
* Ubuntu-22.04    Stopped         2
```

Microsoft 文档说明，可以用 `wsl --list --verbose` / `wsl -l -v` 查看已安装发行版及其 WSL 版本。(Microsoft Learn)

如果 Ubuntu-22.04 不是 VERSION 2，改成 WSL2

如果你看到：

```
Ubuntu-22.04    Stopped    1
```

执行：

```
wsl --set-version Ubuntu-22.04 2
```

并把以后新装的发行版默认设为 WSL2：

```
wsl --set-default-version 2
```

Microsoft 文档说明，`wsl --set-version <distribution name> <versionNumber>` 可以把某个发行版设置为 WSL 1 或 WSL 2，`wsl --set-default-version 2` 可以把新发行版默认设为 WSL2。(Microsoft Learn)

---

7. 设置默认 Ubuntu 发行版

如果你只装了一个 Ubuntu，可以跳过。

如果你电脑里有多个 WSL 发行版，建议设定默认版本：

```
wsl --set-default Ubuntu-22.04
```

以后直接输入：

```
wsl
```

就会进入 Ubuntu 22.04。

Microsoft 文档说明，`wsl --set-default <Distribution Name>` 可以设置默认 WSL 发行版。(Microsoft Learn)

第二步. Ubuntu

第一次进入 Ubuntu 22.04

执行：

```
wsl -d Ubuntu-22.04
```

或者从开始菜单搜索：

```
Ubuntu 22.04
```

第一次启动时，Ubuntu 会让你创建 Linux 用户名和密码。Microsoft 文档说明，首次启动新安装的 Linux 发行版时，会解压文件并要求创建用户账户和密码。(Microsoft Learn)

例如：

```
Enter new UNIX username: yxs
New password:
Retype new password:
```

注意：输入密码时屏幕不会显示字符，这是正常的。

用 root 进入 Ubuntu

在 **PowerShell** 执行：

```
wsl -d Ubuntu-22.04 -u root
```

进入后执行：

```
whoami
```

应该输出：

```
root
```

Microsoft 文档说明，可以用 `wsl --distribution <Distribution Name> --user <User Name>` 指定发行版和用户进入，例如指定 `root`；如果用户不存在才会报错，而 `root` 是 Linux 里默认存在的用户。

把这个 WSL 设置成默认 root + systemd

在 root 里面执行：

```
cat > /etc/wsl.conf <<'EOF'
[boot]
systemd=true

[user]
default=root
EOF

cat /etc/wsl.conf
```

Microsoft 的 WSL 配置文档说明，`/etc/wsl.conf` 是每个发行版自己的配置文件，可以配置 systemd 和默认用户；`[user] default=...` 用来指定 WSL 会话启动时使用哪个用户 。Docker 专用入口也正是用 `-u root` 进入 `/root/workspace` 并启动 Docker 服务

把这个 WSL 设置成默认 root + systemd

进入 Ubuntu 后做基础更新

进入 Ubuntu 终端后，执行：

```
sudo apt update
sudo apt upgrade -y
```

再装一些常用工具：

```
sudo apt install -y curl wget git vim nano unzip ca-certificates gnupg lsb-release
```

确认 Ubuntu 版本：

```
cat /etc/os-release
```

你应该能看到类似：

```
VERSION="22.04.x LTS (Jammy Jellyfish)"
```

为后续 Docker Engine 做准备：启用 systemd

DataChallenging 本地课程环境采用 **WSL Ubuntu 内的 Docker Engine** 运行方式，不依赖 Docker Desktop，所以建议现在就启用 `systemd`。目标是在 WSL 内安装 Docker Engine 与 Compose 插件。

先在 Ubuntu 里检查：

```
ps -p 1 -o comm
```

如果输出已经是：

```
systemd
```

这一步可以跳过。

如果不是，执行：

```
sudo tee /etc/wsl.conf > /dev/null <<'EOF'
[boot]
systemd=true
EOF
```

然后退出 Ubuntu：

```
exit
```

回到 **Windows PowerShell**，执行：

```
wsl --shutdown
wsl -d Ubuntu-22.04/wsl -d Ubuntu
```

再检查：

```
ps -p 1 -o comm=
```

Microsoft WSL 配置文档说明，启用 systemd 需要在 `/etc/wsl.conf` 中加入 `[boot] systemd=true`，然后从 PowerShell 执行 `wsl.exe --shutdown` 重启 WSL 实例。(Microsoft Learn)

第三步.检查最终状态

在 PowerShell 执行：

```
wsl -l -v
wsl --status
```

期望：

```
Ubuntu-22.04    Running 或 Stopped    2
```

进入 Ubuntu：

```
wsl -d Ubuntu-22.04
```

在 Ubuntu 里执行：

```
whoami
pwd
cat /etc/os-release
ps -p 1 -o comm=
```

理想结果：

```
Ubuntu 22.04.x LTS
systemd
```

第四步.最小完整版总结

在 **PowerShell 管理员模式**执行：

```
wsl --update
wsl --list --online
wsl --install -d Ubuntu-22.04
```

重启 Windows 后：

```
wsl -l -v
wsl --set-version Ubuntu-22.04 2
wsl --set-default Ubuntu-22.04
wsl -d Ubuntu-22.04
```

进入 Ubuntu 后：

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y curl wget git vim nano unzip ca-certificates gnupg lsb-release

cat /etc/os-release
```

为后续 Docker Engine 启用 systemd：

```
sudo tee /etc/wsl.conf > /dev/null <<'EOF'
[boot]
systemd=true
EOF

exit
```

回到 PowerShell：

```
wsl --shutdown
wsl -d Ubuntu-22.04
```

再在 Ubuntu 里确认：

```
ps -p 1 -o comm=
```

看到：

```
systemd
```

就说明 **Windows + WSL2 + Ubuntu 22.04 LTS** 基础环境完成。下一步才是在 Ubuntu 里安装 Docker Engine。

🐬 ——— Docker Engine & Docker Compose ——— 🐬

Docker 官方 Ubuntu 安装文档也是先配置 Docker apt 仓库，再安装 `docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin`。(Docker Documentation)

先确认当前状态

在 Ubuntu 里执行：

```
whoami
ps -p 1 -o comm=
cat /etc/os-release
```

理想结果类似：

```
root
systemd
Ubuntu 22.04.x LTS
```

清理可能存在的旧 Docker 包

```
for pkg in docker.io docker-doc docker-compose docker-compose-v2 \
  podman-docker containerd runc; do
  apt-get remove -y $pkg
done
```

更新 apt，并安装基础依赖

```
apt-get update

apt-get install -y ca-certificates curl gnupg
```

第二步. 走官方源/Ubuntu仓库

策略一：Docker官方镜像源安装 🐳

添加 Docker 官方 GPG key

```
install -m 0755 -d /etc/apt/keyrings

curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc

chmod a+r /etc/apt/keyrings/docker.asc
```

添加 Docker 官方 apt 源

```
cat > /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

apt-get update
```

这一步对应你文档里的 Docker 官方仓库安装流程，核心就是写入 `/etc/apt/sources.list.d/docker.sources`，然后 `apt-get update` 。

安装 Docker Engine + Compose 插件

```
apt-get install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
```

Docker 官方文档给出的最新版本安装包名也是这一组：`docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin` 。(Docker Documentation)

启动 Docker 服务并设置开机启动

```
systemctl enable --now docker

systemctl status docker --no-pager
```

看到类似：

```
active (running)
```

就说明 Docker daemon 已启动。Docker 官方文档也说明安装后应检查 `systemctl status docker`，如果没运行就用 `systemctl start docker` 启动。(Docker Documentation)

验证 Docker 和 Compose

```
docker version
docker compose version
docker ps
docker run --rm hello-world
```

你文档里的验证项也是： `docker version` 同时看到 Client 和 Server， `docker compose version` 能输出版本号， `docker ps` 正常显示表头， `hello-world` 输出成功信息 。Docker 官方也推荐用 `docker run hello-world` 验证安装成功。(Docker Documentation)

总结：可执行版脚本

```
whoami
ps -p 1 -o comm=
cat /etc/os-release

for pkg in docker.io docker-doc docker-compose docker-compose-v2 \
  podman-docker containerd runc; do
  apt-get remove -y $pkg
done

apt-get update
apt-get install -y ca-certificates curl gnupg

install -m 0755 -d /etc/apt/keyrings

curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc

chmod a+r /etc/apt/keyrings/docker.asc

cat > /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

apt-get update

apt-get install -y docker-ce docker-ce-cli containerd.io \
```

```
  docker-buildx-plugin docker-compose-plugin

systemctl enable --now docker

systemctl status docker --no-pager

docker version
docker compose version
docker ps
docker run --rm hello-world
```

策略二：使用 Ubuntu 仓库安装 Docker🐳

你现在要走的是 **Ubuntu 仓库版 Docker**，不添加 Docker 官方源，也不写 `/etc/apt/sources.list.d/docker.sources`。

Ubuntu 官方包页面也显示 Ubuntu 22.04 Jammy 仓库里有 `docker.io` 包，属于 `universe` 组件。(Ubuntu 软件包搜索)

先执行这一整段：

```
# 1. 如果之前添加过 Docker 官方源，先删除，避免混用
rm -f /etc/apt/sources.list.d/docker.list
rm -f /etc/apt/sources.list.d/docker.sources
rm -f /etc/apt/keyrings/docker.asc
rm -f /etc/apt/keyrings/docker.gpg

# 2. 清理 apt 缓存并更新
apt-get clean
apt-get update

# 3. 确保 universe 仓库可用
apt-get install -y software-properties-common
add-apt-repository -y universe

# 4. 再更新一次
apt-get update

# 5. 查看 Ubuntu 仓库里 Docker 包是否可用
apt-cache policy docker.io docker-compose-v2 docker-compose

# 6. 使用 Ubuntu 仓库安装 Docker 和 Compose v2
apt-get install -y docker.io docker-compose-v2

# 7. 启动 Docker 服务
```

```
systemctl enable --now docker

# 8. 验证
systemctl status docker --no-pager
docker version
docker compose version
docker ps
```

Docker 官方文档也提醒，`docker.io`、`docker-compose`、`docker-compose-v2` 这类包是 Linux 发行版提供的包，和 Docker 官方源里的 `docker-ce` 等包不要混装；所以你现在选择 Ubuntu 仓库路线，就不要再同时保留 Docker 官方 apt 源。(docs.docker.com)

📊 DataChallenging 环境与课程Project

1.阶段一：Task-Agent 初始方案

设计 Task 和 Agent

你需要完成：

- 设计一道数据分析题；
- 准备数据文件；
- 编写 `reference.py`；
- 编写 Agent；
- 本地运行测试；
- 生成提交文件。

```
  bin/datac --help                                  # 查看命令
  bin/datac doctor --workspace /workspace           # 环境自检（零网络）
  bin/datac init --target /workspace                # 模板初始化 workspace
  # 编辑 workspace/task.json 与 workspace/agent.py
  bin/datac run --workspace /workspace              # 运行
  bin/datac submission build \
  --input /workspace \
  --student-id 4725200597 \
  --phase P1

bin/datac submission validate \
--input /workspace/submission/bigdata_p1_4725200597.zip \
--phase P1
```

2.阶段二：Task-Agent 最终方案

匿名题目分析

你会收到其他同学设计的匿名 Task。

你的任务通过真实运行和分析理解数据分析任务及 Agent 的能力边界。

需要完成：

1. 阅读题目；
2. 理解数据；
3. 尝试运行 Agent；
4. 分析运行结果；
5. 判断题目是否存在问题；
6. 判断 Agent 为什么成功或失败；
7. 提出改进建议。

重点理解：

- 什么样的问题容易被模型理解；
- 什么样的数据组织容易产生错误；
- 哪些字段或条件容易被误解；
- Agent 为什么会失败；
- Task 怎样修改后会更清楚、更可验证。

优化 Task 和 Agent

根据：

- 自己的测试结果；
- 匿名题目分析结果；
- 收到的题目分析意见；

完成：

```
Task v1  →  Task v2
Agent v1 →  Agent v2
```

最终目标是让 Task 和 Agent 能够作为你的最终方案。

---

3. 启动 DataChallenging 本地环境

3.1 启动 Docker

进入 WSL Ubuntu 后执行：

```
systemctl start docker.service
systemctl status docker --no-pager
```

如果看到：

```
active (running)
```

说明 Docker 服务正常。

3.2 进入课程本地部署目录

课程离线包、Docker Compose 目录和镜像文件名可能随课程发布版本变化，因此**以教师实际发放的目录和文件名为准**。

例如：

```
cd /root/BigData-v10.1.XX
```

确认当前目录中存在 Compose 配置：

```
ls
```

应能看到：

```
docker-compose.yml
```

image文件夹

3.3 第一次启动

如果课程环境还没有创建容器，执行：

```
docker load -i images/bigdata-student-10.1.16.image.tar
docker compose up -d
docker compose ps
```

<课程镜像文件>.tar的位置存放在当前工程项目BigData-v10.1.XX下的**images**文件夹

如果课程提供的是离线 Docker 镜像，应先按照教师提供的文件名导入：

```
docker load -i <课程镜像文件>.tar
```

> 不要自行猜测镜像名或版本号。课程升级后，以实际发放包中的 Compose 文件和镜像文件为准。

3.4 日常快速启动

如果只是关机、重启 WSL 或容器停止：

```
cd /root/BigData-v10.1.XX

docker compose up -d
docker compose ps
```

如果之前执行过：

```
docker compose down
```

容器已经被删除，则重新执行：

```
docker compose up -d
docker compose ps
```

3.5 一键启动

为了方便加载镜像，本系统提供直接一键启动Docker

```
scripts/install.sh
```

执行结果

```
[install] loading bigdata-student:10.1.XX image ...
Loaded image: bigdata-student:10.1.XX
[install] starting container ...
[+] Running 1/1
 ✓ Container bigdata-student  Running


                          0.0s
[install] doctor:
[+] Running 1/1
 ✓ Container bigdata-student  Running0.0s
[doctor] PASS  python_version  3.12.15
[doctor] PASS  runtime_policy  policy/runtime_policy.json 合法
[doctor] overall=PASS
[install] done. 用 bin/datac --help 查看课程 CLI。
```

---

4. 检查 DataChallenging CLI

进入课程的工作区目录：

```
cd /root/workspace/
```

执行：

```
bin/datac --help
```

成功标志：

- 命令可以正常执行；
- 可以看到课程 CLI 的帮助信息；
- 没有出现 `command not found`；
- 后续可以执行 `run` 和 `build`。

如果提示：

```
codesetarena: command not found
```

说明当前 shell 中没有找到课程 CLI。此时应优先检查：

1. 是否进入了正确的课程环境；
2. 是否需要进入课程容器执行；
3. 是否需要激活课程提供的 Python/Conda 环境；
4. 教师发布包中是否提供了 CLI 安装脚本。

不要自行修改课程接口名称。

---

5. 明确工作区目录

`workspace` 作为两个阶段工作区 + 两个原始恢复包。

课程镜像已经提前准备好两个阶段的工作目录，不再需要手动创建 workspace，也不需要第一次使用时手动解压模板**。

```
workspace/
│
├── .env
├── .env.example
├── README.md
│
├── phase1/                    # 阶段一默认工作区，镜像中已提前解压
│   ├── task.json
│   ├── data.csv
│   ├── reference.py
│   ├── agent.py
│   └── metadata.json          # 强调必须填写自己的学号；"student_id": 学号
│
├── phase2/                    # 阶段二默认工作区，镜像中已提前解压
│   ├── task.json
│   ├── data.csv
│   ├── reference.py
│   ├── agent.py
│   ├── metadata.json          # 强调必须填写自己的学号；"student_id": 学号
│   ├── review-comments.json
│   └── 实验报告模板.docx
│
├── exposure/
│   ├── assignment.json
│   │
│   ├── AT028/
│   │   ├── task.json
│   │   ├── data.csv
│   │   └── reference.py
```

```
│   │
│   ├── AT035/
│   │   ├── task.json
│   │   ├── data.csv
│   │   └── reference.py
│   │
│   └── ...
│
├── runs/
│   ├── phase1/
│   ├── phase2/
│   └── exposure/
│
└── store/
    ├── packages/
    └── builds/
```

内容	作用
`phase1/`	阶段一工作目录，已经提前解压好 Phase 1 默认模板
`phase2/`	阶段二工作目录，已经提前解压好 Phase 2 默认模板
`BigData-Phase1.zip`	Phase 1 原始模板备份，用于恢复阶段一工作区
`BigData-Phase2.zip`	Phase 2 原始模板备份，用于恢复阶段二工作区

课程已经为两个阶段分别准备独立工作区：

```
phase1/     第一次正式提交使用
phase2/     最终提交使用
```

两个目录相互独立。

Phase 1

阶段一直接进入：

```
cd workspace/phase1
```

`phase1/` 中已经提供完整的默认 Task + Agent 工作模板，

Phase 2

阶段二直接进入：

```
cd workspace/phase2
```

`phase2/` 同样由镜像提前提供完整的 Phase 2 默认模板。

镜像同时保留两个阶段的原始 ZIP 模板，就是为了让方便随时能够恢复。

> `phase1_template.zip` 和 `phase2_template.zip` 是课程提供的原始模板备份。
> 如果在实验过程中误删文件、修改错误或希望重新开始，可以使用对应 ZIP 恢复工作目录。：

不同阶段只操作自己的目录，命令必须显式指定 `--phase` 。

建议每个阶段都在自己的工作区内开发、测试和提交，两个阶段互不覆盖。

测试标准答案：

```
bin/datac reference run --phase phase1
```

运行自己的 Agent：

```
bin/datac run --phase phase1
```

注明自己的学号，请在：

```
metadata.json
```

中填写自己的学号：

```
{
  "student_id": "你的学号"
  "task_id": "TASK_你的学号",
  "agent_id": "AGENT_你的学号"
}
```

强调要填写student_id，task_id和agent_id

workspace/目录下执行：

```
unzip -o BigData-Phase1.zip
```

就会直接覆盖,避免产生这种错误结构：

```
workspace/phase1/phase1/
```

推荐完整重置：

```
cd workspace
rm -rf phase1
unzip BigData-Phase1.zip
```

Phase 2 同理：

```
cd workspace
rm -rf phase2
unzip BigData-Phase2.zip
```

这样才是真正恢复成课程镜像最初提供的状态。

6. 编写 Task

Task = 题干 + CSV 数据 + 参考程序（参考答案）

```
phase1/
└── 20250001/
    ├── task/
    │   ├── task.json
    │   ├── data.csv
    │   └── reference.py
    │
    ├── agent/
    │   └── agent.py
    │
    └── metadata.json
```

题干task.json 就是一道**可执行、可验证的数据分析题**。

基本格式如下：

```
{
  "description": "一句话概括你的任务。",
  "task_statement": "完整说明数据、计算要求和最终输出格式。",
  "hypothesized_failure_mode": "FM_AGGREGATION",
  "answer": 30
}
```

`task_statement`：完整说明这道题到底要做什么

可以按照下面的顺序写：

```
① 数据说明
② 筛选或处理规则
③ 计算目标
④ 输出要求
```

不要只追求题目“难”，更重要的是题目能够被准确理解和稳定评测。

`hypothesized_failure_mode`：填写失败模式

必须填写系统规定的完整 Failure Mode Key，例如：

```
FM_REQUIREMENT
FM_SCHEMA
FM_DATA_TYPE
FM_FILTER_SORT
FM_JOIN
FM_AGGREGATION
FM_TIME_TIMEZONE
FM_MISSING_VALUE
FM_OUTLIER_BOUNDARY
FM_DUPLICATE_RECORD_HANDLING
FM_PLANNING_DECOMPOSITION
FM_VERIFICATION
FM_NUMERICAL_PRECISION_ROUNDING
FM_PROMPT_INJECTION
```

`answer`：填写这道题的标准答案

`answer` 是你执行自己写的reference.py 得到的正确答案。

正确答案执行指令

阶段一固定在 phase1 跑，最终阶段固定在 phase2 跑

```
bin/datac reference run --phase phase1
#如果更换了题目，强调每个阶段都在自己的工作区内开发、测试和提交，两个阶段互不覆盖。
bin/datac reference run --phase phase2
```

最终结果类似：

```
============================================================
BigData Reference Runner (v10.1.16)
============================================================
Phase: phase1
Status: PASS

{
  "answer": 30
}
============================================================
reference: phase1/reference.py
data:      phase1/data.csv
```

---

7. 准备数据

课程数据通常使用 CSV 文件。

例如：

```
data.csv
```

准备数据时应确认：

- CSV 格式正确；
- 表头存在且字段名称清楚；
- 数据类型可以被 Python 正常读取；
- 缺失值的含义明确；
- Task 中引用的字段真实存在；
- 数据足以支持题目计算；
- 不依赖个人机器的绝对路径。

可以先用 Python 快速检查：

```
python - <<'PY'
import pandas as pd

df = pd.read_csv("data.csv")

print(df.head())
print(df.columns.tolist())
print(df.dtypes)
print(df.shape)
PY
```

如果可以正常输出，说明 CSV 至少可以被 pandas 正常读取。

---

8. 编写程序 ⭐ ⭐ ⭐

程序一：标准答案程序 reference.py

`reference.py` 用来计算该 Task 的正确答案。

它不是"答案说明"，而是**可执行的标准答案程序**。

要求：

- 可以独立运行；
- 能读取 Task 所需数据；
- 输出唯一答案；
- 结果稳定；
- 不依赖个人环境；
- 不访问本机私有文件；
- 不使用随机结果作为最终答案。

最重要的是：

```
同一份 Task + 同一份 Data
        ↓
reference.py
        ↓
唯一 Ground Truth
```

编写完成后先自行执行：

```
python reference.py
```

确认没有异常，并记录最终输出的数据类型和格式。

程序二：编写 Agent程序agent.py

Agent 是你编写的自动解题程序。

Agent 应根据：

```
Task
 +
Data
```

完成数据分析并返回最终答案。

课程接口**以当前课程模板为准**，不要自行修改函数名、输入协议或输出协议。

Agent 的重点不是写死某一道题的答案，而是完成课程模板要求的数据读取、任务理解、分析和返回过程。

建议至少检查：

- Agent 是否能读取数据；
- 是否正确理解问题；
- 是否正确识别字段；
- 是否正确处理数据类型；
- 是否执行了正确的统计/分析逻辑；
- 最终输出是否符合课程要求的数据类型。

标准答案与解题 Agent两个程序要求

在课程的两个阶段当中，你会同时看到：

```
├── task.json
├── data.csv
├── reference.py
└── agent.py
```

`reference.py` 和 `agent.py` 都会由平台执行，但它们承担的是两种完全不同、又相互配合的角色。

最简单地记：

> **reference.py 是“标准答案”，agent.py 是“智能体的解题方法”。**

1. 两个程序文件分别是做什么的?

例如，Task 是：

> 根据 `sales.csv`，计算销售额最高的商品类别。

那么：

```
task.json
    ↓
告诉系统“要算什么”

data.csv
    ↓
提供真实数据

reference.py
    ↓
按照确定的方法算出正确答案
    ↓
Gold Answer

agent.py
    ↓
让你的 Agent 自己理解题目、分析 CSV
    ↓
Agent Answer
```

最后平台比较：

```
Agent Answer
      vs
Gold Answer
```

判断 Agent 是否正确。

2. 标准答案程序(`reference.py`)说明

reference.py：这道题的“标准答案”

`reference.py` 的任务只有一个：

> **给定数据，稳定地计算出这道 Task 的正确结果。**

它不是智能 Agent，也不需要“理解”题目。

典型形式可以理解为：

```
def reference(df):
    ...
    return {"answer": ...}
```

它只依赖：

```
CSV 数据
```

然后严格按照题目要求进行：

```
筛选
→ 计算
→ 聚合
→ 排序
→ 输出正确答案
```

reference.py 最重要的要求

它必须：

```
简单
明确
确定
可验证
```

也就是说：

> 同一份数据运行多少次，都应该得到完全相同的答案。

因此 `reference.py` ：

- 不使用 LLM；
- 不依赖随机行为；
- 不需要复杂容错；
- 不应该隐藏错误；
- 应直接实现题目规定的计算逻辑。

例如：

```
题目：
统计 status="failed" 的记录数

reference.py：
df[df["status"] == "failed"].shape[0]
```

越直接越好。

3. 解题程序文件（`agent.py`）说明

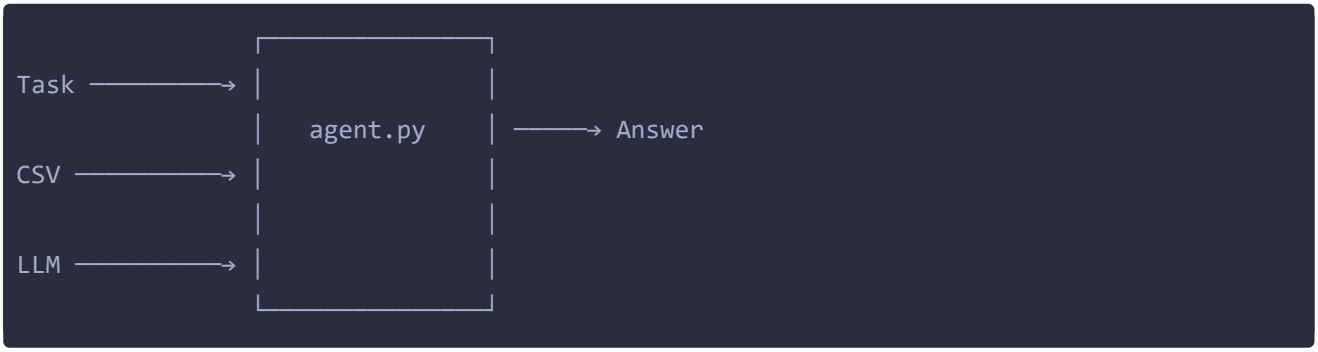
agent.py：你真正要设计和改进的东西

`agent.py` 则完全不同。

它面对的不只是自己设计的这一道题，而应该能够处理：

> **不同题目 + 不同 CSV 数据。**

可以把它理解成：



Agent 需要自己完成：

```
理解问题
    ↓
理解数据字段
    ↓
制定分析步骤
    ↓
读取和处理数据
    ↓
进行计算
    ↓
检查结果
    ↓
输出最终答案
```

因此它才是这门课程真正要测试的对象。

4. 课程要求设计程序文件

在阶段一，你同时扮演两个角色：

```
Task Designer
+
Agent Developer
```

所以你需要完成：

```
Task v1
├── task.json
├── data.csv
└── reference.py

Agent v1
└── agent.py
```

也就是说，你既要：

> **出一道数据分析题**

又要：

> **给这道题提供一份正确的标准答案**

同时还要：

> **设计一个 Agent 来尝试解决数据分析任务。**

Phase 2 中，同学会收到来着其他人的匿名 Task。

他们应该看到的是：

```
题目
+
数据
```

而不能看到：

```
标准答案
```

否则挑战就失去意义了。

因此概念上：

```
你的 Task
├── task.json       → 可以匿名分发
├── data.csv        → 可以匿名分发
└── reference.py    → 私有，不发给 Reviewer
```

Reviewer 会使用：

```
自己的 agent.py
```

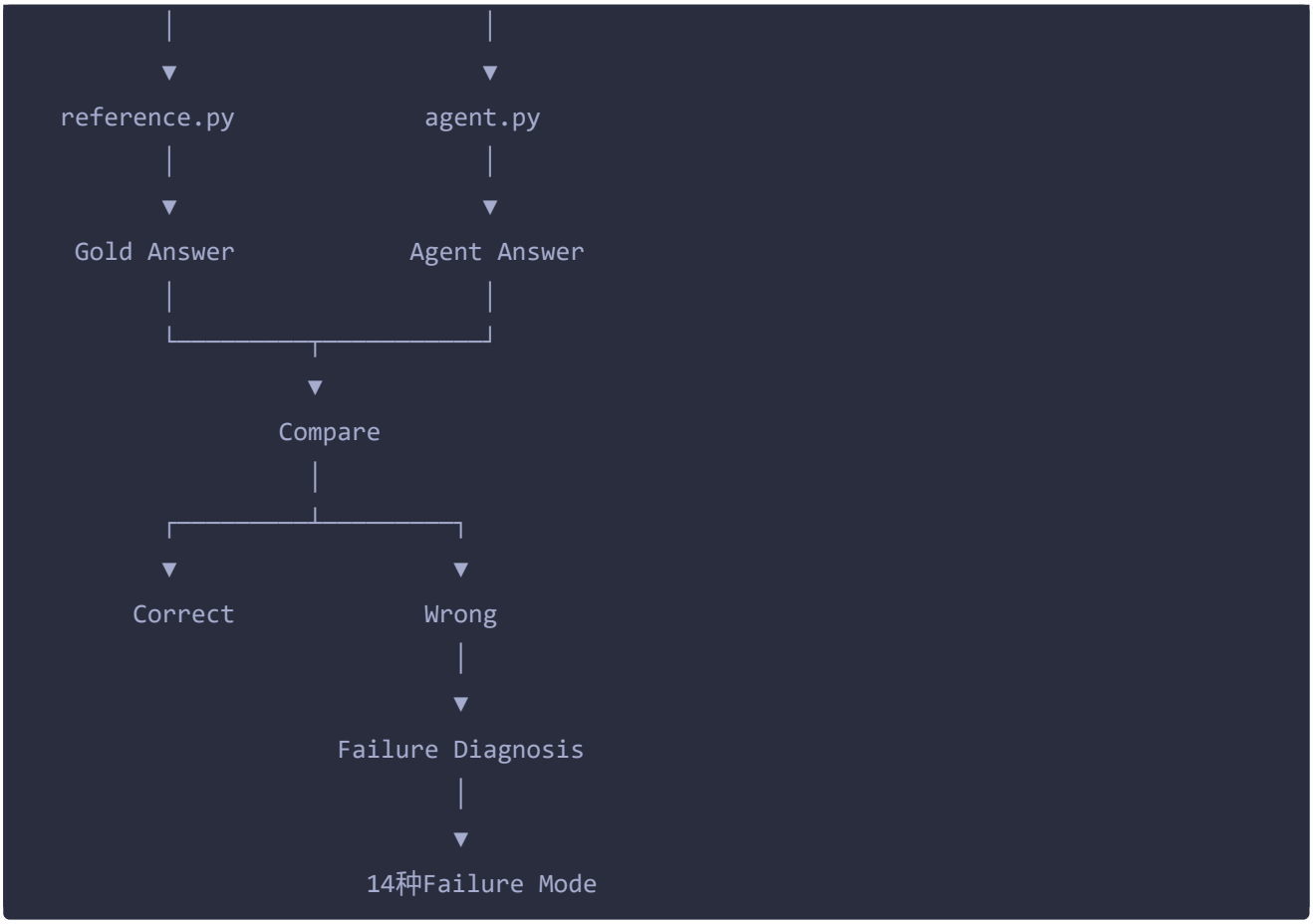
来解决你的匿名 Task。

因此：

```
你的 reference.py
        ↓
教师 / 平台私有 Gold

别人的 agent.py
        ↓
挑战你的匿名 Task
```





reference.py

目标是：

> **把正确答案写清楚。**

应该：

```
短
直接
确定
容易人工检查
```

不要：

```
调用 LLM
加入随机逻辑
针对运行环境做复杂判断
大量 try/except 掩盖错误
```

可以把它理解成：

> **老师手里的标准答案。**

agent.py

目标是：

> **构建一个尽可能通用的数据分析 Agent。**

应该考虑：

```
如何理解 Task
如何理解 CSV
如何规划分析步骤
如何调用 LLM
如何验证计算结果
如何控制 Token
如何满足输出格式
如何处理失败
```

尤其不能：

```
if 当前 data.csv:
    return 已知答案
```

也就是不能针对自己的数据直接写死答案。

因为第二阶段它要面对的是：

```
10 个完全陌生的 Task
```

写死自己 Task 的答案没有任何意义。

两者最大的运行差异

维度	`reference.py`	`agent.py`
输入	数据	Task + 数据 + LLM
LLM	禁止使用	可以使用
确定性	必须确定	可以存在 LLM 随机性
Token 消耗	不使用 Token Budget	LLM 调用消耗课程 Token Budget
资源约束	普通确定性执行	受时间、内存、Token 等限制
失败代表什么	Task 的标准答案实现有问题	Agent 的数据分析能力出现问题

维度	`reference.py`	`agent.py`
是否是课程主要评价对象	否	**是**

---

10. Failure Mode（失败模式）

Failure Mode 表示：

> 你认为 Agent 在完成该 Task 时最可能出现哪类错误。

例如：

- 需求理解错误；
- 字段理解错误；
- 数据类型错误；
- 过滤条件错误；
- 分组或聚合错误；
- 统计计算错误；
- 输出格式错误。

填写时必须使用课程提供的 **FM 编号**。

FM 编号	中文名称	含义说明
`FM_REQUIREMENT`	需求理解偏差	Agent 误解任务目标、条件或输出要求。
`FM_SCHEMA`	Schema 理解错误	Agent 错误理解字段含义、主键、字段关系或数据粒度。
`FM_DATA_TYPE`	数据类型处理错误	Agent 错误处理数值、字符串、日期、布尔值等数据类型。
`FM_FILTER_SORT`	筛选或排序错误	Agent 使用了错误的筛选条件、排序字段、排序顺序或方向。
`FM_JOIN`	Join / 关联错误	Agent 使用了错误的关联字段或 Join 方式。
`FM_AGGREGATION`	聚合口径错误	Agent 在求和、平均、分组统计等过程中使用了错误的计算口径。
`FM_TIME_TIMEZONE`	时间与时区错误	Agent 错误处理日期、时间范围、时间窗口或时区。
`FM_MISSING_VALUE`	缺失值处理错误	Agent 错误识别或处理 NaN、null、空字符串等缺失数据。

FM 编号	中文名称	含义说明
`FM_OUTLIER_BOUNDARY`	异常值或边界处理错误	Agent 错误处理异常值、阈值或边界条件。
`FM_DUPLICATE_RECORD_HANDLING`	重复记录处理错误	Agent 错误识别、保留、删除或统计重复记录。
`FM_PLANNING_DECOMPOSITION`	规划与分解错误	Agent 的任务拆解或执行步骤顺序不合理。
`FM_VERIFICATION`	验证不足	Agent 未对中间结果或最终结果进行必要检查。
`FM_NUMERICAL_PRECISION_ROUNDING`	数值精度与舍入错误	Agent 在数值精度、小数位或舍入方式上处理错误。
`FM_PROMPT_INJECTION`	提示注入或不可信数据	Agent 被数据中的不可信指令或文本影响了正常分析行为。

FM 编号作为系统Failure Mode 的唯一标识，强调不要自行创建与课程编号体系不一致的新编号。

Failure Mode 的作用不是给 Agent “判分”，而是帮助你在设计阶段形成一个可验证的失败假设：

```
我设计的 Task
      ↓
Agent 可能在哪里出错?
      ↓
实际运行时是否真的发生?
      ↓
为什么发生?
      ↓
Task / Agent 应该怎样改进?
```

11. 提交Task-Agent 初始设计

11.1 测试 Task-Agent 初始设计

本地运行测试指令:

```
bin/datac run \
  --workspace /workspace
```

它对应：

```
task.json + data.csv
        ↓
agent.py
        ↓
本地 Runtime 执行
        ↓
生成运行结果
```

11.2 构建 Phase 1 提交包

在课程目录执行：

```
  bin/datac submission build \
  --input /workspace \
  --student-id 4725200597 \
  --phase P1
```

系统会完成类似流程：

```
检查 Task 文件
      ↓
读取数据
      ↓
运行 Agent
      ↓
计算标准答案
      ↓
比较 Agent Output 与 Ground Truth
      ↓
生成运行结果
```

11.3 本地验证非常重要

完成本地测试后，**必须在提交前执行一次正式验证**：

```
bin/datac submission validate \
  --input /workspace/submission/bigdata_p1_4725200597.zip \
  --phase P1
```

这一步是正式提交前的最后一次自检。

因此，只要你本地使用最终准备提交的 ZIP 执行：

```
local_validation = PASS
```

那么助教收到**同一个 ZIP 文件**后，在课程环境一致的情况下，通常也会得到相同的验证结果。

所以，不要等提交以后才让助教帮你发现问题。提交前，自己就可以先发现：

```
Task 文件是否缺失
Data 文件是否正确
reference.py 是否能运行
Agent 是否能运行
学号是否一致
提交阶段是否正确
文件结构是否符合要求
必填字段是否完整
```

11.4 本地验证

本地测试完成后执行：

```
bin/datac submission validate \
--input /workspace/submission/bigdata_p1_4725200597.zip \
--phase P1
```

系统会检查提交内容并生成课程要求的最终提交文件。

不要手工修改 ZIP，也不要自己重新压缩来绕过检查。

请务必验证“最终要提交的那个 ZIP”，不要只验证工作区。助教收到的也是这个 ZIP，助教侧会用同一套验证程序重新检查。

12. 理解运行结果

本地测试结果可以先分成三类。

(1)正确完成

Agent 正常执行，并且最终输出与标准答案一致。

重点不是只看“运行成功”，还要确认与reference.py执行结果：

```
Agent Output == Ground Truth
```

Agent 执行结果：Agent Output

reference.py执行结果：Ground Truth

---

(2)答案错误

Agent 程序正常运行，但最终答案错误。

此时重点分析：

- 是否理解错题目；
- 是否选择错字段；
- 是否漏掉过滤条件；
- 是否出现数据类型转换错误；
- 是否使用了错误的统计方法；
- 是否输出格式不符合要求。

---

(3)运行失败

程序或模型调用没有正常完成。

常见检查方向：

- Python 异常；
- 文件路径错误；
- 数据文件不存在；
- API 调用失败；
- Agent 接口不符合模板；
- 输出没有正常返回；
- Docker / 本地课程服务没有启动。

---

13. 提交Task-Agent 最终设计

任务一：匿名Tasks分析BigData

阶段二收到的是其他同学设计的匿名 Task。

> 通过真实 Task 和真实 Agent 运行结果，分析数据分析任务设计与 Agent 能力边界。

假设老助教发放下来的包是：

```
BigData_Exposure_R000001.tar.gz
```

1 导入匿名Tasks

```
bin/datac exposure import \
  --package /workspace/BigData_Exposure_R000001.tar.gz \
  --workspace /workspace
```

检查：

```
bin/datac exposure status \
  --workspace /workspace
```

目标：

```
tasks=10/10
done=0
```

进入workspace工作区,查看exposure文件夹

此时会看到ATxxx（示例条目），强调每个AT XXX对应的是一个匿名Tasks

2 测试匿名 Task

用自己的 Agent v1 跑 10 个匿名任务

先确认 `.env` 存在：

```
ls -lh workspace/.env
```

测试发放的 10 个匿名任务：

```
bin/datac exposure run \
  --workspace /workspace
```

跑完检查：

```
bin/datac exposure status \
  --workspace /workspace
```

目标：

```
[exposure] status=READY tasks=10/10 done=10
```

3 分析匿名 Task

a.题目要求是否清楚?

检查：

- 目标是否明确；
- 是否存在多种解释；
- 是否明确要求计算什么；
- 是否可以判断最终答案类型。

b 数据是否支持问题?

检查：

- 所需字段是否存在；
- 数据是否足够；
- 字段含义是否清楚；
- 是否存在无法从数据中确定的信息。

最终要求填写review-comments.json

强调review-comments.json里面必须填写以下信息

```
{
  "ATxxx（示例条目，请整条替换）": {
    "task_id": "ATxxx（示例条目，请整条替换）",
    "score": 0,
    "comments": "模板条目：每道分配给你的题各写一条；score 为 1~5 的整数；comments 为非空评语。导入正式分配后请整条替换本示例。"
  }
}
```

填写内容严格按照每个AT XXX对应进行填写，否则最终阶段是无法正常导出提交包

任务二：Task-Agent 方案敲定

1.敲定Task 方案

Phase 2 不另外创建 `task_v2.json` 。

仍然修改原来的：

```
workspace/task.json
workspace/data.csv
```

例如直接编辑：

```
nano workspace/task.json
```

如果 Task v2 需要调整数据：

```
nano workspace/data.csv
```

也可以直接用 VS Code 修改。

2.敲定Agent 方案

Phase 2 不另外创建 `agent_v2.py` 。

仍然修改：

```
workspace/agent.py
```

例如直接编辑：

```
nano workspace/agent.py
```

逻辑就是：

```
Agent v1
   ↓
跑 10 个陌生任务
   ↓
发现失败原因并且分析
   ↓
修改 agent.py
   ↓
Agent v2
```

3 测试 Task-Agent 最终方案

本地运行测试指令:

```
bin/datac run --workspace /workspace/phase2
```

它对应:

```
task.json + data.csv
        ↓
agent.py
        ↓
本地 Runtime 执行
        ↓
生成运行结果
```

4 构建 Phase 2 提交包

在课程目录执行:

```
bin/datac submission build --input /workspace/phase2 --student-id 4725200597
```

目标:

```
[submission] build status=built
local_validation=PASS
error_code=None
```

最终得到:

```
/workspace/phase2/submission/bigdata_p1_20260001.zip
```

也就是：

```
bigdata_p1_<student-id>.zip
```

5 本地验证

本地测试完成后执行：

```
bin/datac submission validate --input
/workspace/phase2/submission/bigdata_p1_4725200597.zip
```

系统会检查提交内容并生成课程要求的最终提交文件。

不要手工修改 ZIP，也不要自己重新压缩来绕过检查。

请务必验证“最终要提交的那个 ZIP”，不要只验证工作区。助教收到的也是这个 ZIP，助教侧会用同一套验证程序重新检查。

任务三：review-comments 和 实验报告

1.Review Comments

Phase 2 中，要求需要对自己收到的每个匿名 Task 完成评审，并将所有评审结果统一保存到：

```
review-comments.json
```

`review-comments.json` 必须采用 **Key-Value 字典结构**：

```
Task ID
   ↓
作为唯一 Key
   ↓
对应这一道 Task 的完整评审结果
```

例如：

```
{
  "AT043": {
    "score": 4,
```

```
    "comments": "任务目标清晰，但边界条件说明不足。"
  },
  "AT048": {
    "score": 5,
    "comments": "任务设计完整，数据与问题匹配，具有较好的分析挑战性。"
  },
  "AT061": {
    "score": 3,
    "comments": "任务可以完成，但部分字段含义不够明确。"
  }
}
```

要求必须填写什么?

对于收到的**每一道匿名 Task**，至少填写两项：

字段	含义
`score`	对该 Task 的评分
`comments`	对该 Task 的具体评语

本课程实际发放的是 10 个匿名 Task，那么最终：

```
review-comments.json

必须有 10 个 Task ID Key
          ↓
每个 Task ID 恰好出现一次
          ↓
每个 Key 对应一组
score + comments
```

> **Review Comments：**
> 对每个收到的匿名 Task 完成评分和评语，并统一保存到 `review-comments.json`。文件采用字典结构，以匿名 `Task ID` 作为唯一 Key，Value 保存该 Task 的 `score` 和 `comments`。

2.实验报告

实验报告不要求大量展示代码和运行日志，重点说明 Task 是怎么设计的、Agent 是怎么设计的、从 v1 到 v2 为什么改或为什么不改，

以及通过本次实验获得了哪些经验和后续优化方向。

一、Task 设计与迭代

这一部分只需要说明**你的 Task 是怎么设计的，以及从 v1 到 v2 有没有变化**。

1. Task v1 设计思路

说明你第一阶段设计的 Task。

重点回答：

- 这个 Task 要解决什么数据分析问题；
- 为什么选择这个问题；
- 使用了什么数据；
- 主要考察哪一种或哪几种数据分析能力；
- 你希望这个 Task 能够暴露 Agent 的什么能力边界。

不需要大篇幅粘贴代码，也不需要逐字段解释 `task.json`。

可以围绕：

> **问题是什么 → 为什么这样设计 → 想考察什么能力**

来写。

---

2. Task v2 设计思路

说明第二阶段最终提交的 Task。

重点回答：

- Task v2 最终是怎样设计的；
- 与第一阶段相比，设计目标是否发生变化；
- 为什么最终形成现在这个版本。

如果 Task v2 与 Task v1 完全相同，也可以。

此时必须明确写：

> **Task v2 与 Task v1 保持一致，本阶段未对 Task 进行修改。**

并简单说明为什么没有修改。

---

3. Task v1 → Task v2 的改变

如果进行了修改，直接说明：

```
Task v1
   ↓
发现的问题 / 得到的启发
   ↓
修改内容
   ↓
Task v2
```

例如可以写：

项目	Task v1	Task v2	修改原因
问题设计	原设计	修改后设计	为什么修改
数据设计	原数据特点	修改后特点	为什么修改
难点设计	原能力点	新能力点	为什么修改

重点不是"改了多少"，而是：

> **为什么改。**

如果没有修改，则直接写：

> 经第二阶段测试与分析后，未发现当前 Task 存在必须调整的问题，因此 Task v2 保留 Task v1 的设计。

二、Agent 设计与迭代

这一部分同样只讲：

> **Agent 是怎么设计的，以及从 v1 到 v2 有没有改变。**

1. Agent v1 设计思路

说明第一阶段 Agent 的总体设计。

重点回答：

- Agent 如何理解 Task；
- Agent 如何使用 CSV 数据；
- Agent 如何进行数据分析；

- 是否使用 LLM；
- 大致的解题流程是什么；
- 你设计这个 Agent 时最关注什么能力。

例如可以概括成：

```
读取 Task
   ↓
理解题目要求
   ↓
分析 CSV Schema
   ↓
规划分析步骤
   ↓
执行数据分析
   ↓
检查结果
   ↓
输出答案
```

不需要把整个 `agent.py` 代码粘贴到实验报告。

实验报告要说明的是：

> **你的 Agent 为什么这样设计。**

---

2. Agent v2 设计思路

说明第二阶段最终 Agent 的设计。

重点回答：

- Agent v2 最终采用什么思路；
- 是否针对 Phase 2 中观察到的问题做了调整；
- 最终希望解决 Agent v1 的什么不足。

如果没有修改，也完全允许。

此时必须明确写：

> **Agent v2 与 Agent v1 保持一致，本阶段未对 Agent 进行修改。**

同时简单说明你的判断理由。

3. Agent v1 → Agent v2 的改变

如果进行了修改，重点说明：

```
Agent v1 出现什么问题
        ↓
你如何判断问题原因
        ↓
采取什么修改
        ↓
希望 Agent v2 改善什么
```

例如：

发现的问题	Agent v1	Agent v2	改进目的
Schema 理解不稳定	原策略	新策略	提高字段理解
缺少结果验证	直接输出	增加检查	降低错误答案
分析规划不足	一次完成	分步骤分析	提高复杂任务稳定性

这里也不要求一定修改。

如果没有修改，可以直接说明：

> 根据第二阶段的运行与分析结果，没有发现足够明确的问题需要修改 Agent，因此最终 Agent v2 保留 Agent v1 的实现。

三、总结与反思

最后只需要进行总体总结，不再增加新的技术章节。

重点回答三个问题：

1. 这次实验你学到了什么?

例如：

- 对数据分析 Task 设计有什么新的认识；
- 对 Agent 能力边界有什么认识；
- 什么样的问题容易让 Agent 失败；
- 如何判断一个 Agent 是否真正具有通用数据分析能力。

2. 哪些设计经验以后可以继续使用？

例如：

```
Task 要有清晰目标
Reference 要可靠
Agent 要先理解 Schema
复杂任务需要分解
结果必须验证
```

重点总结**可迁移的经验**。

3. 如果继续迭代，下一步准备优化什么？

这里写未来方向即可，例如：

- 提高 Agent 对复杂 Schema 的理解；
- 增强 Join 能力；
- 增加结果验证机制；
- 改进任务规划；
- 设计更有区分度的 Task；
- 增加边界数据和异常数据；
- 提高 Agent 的稳定性和 Token 使用效率。

最终实验报告结构

因此整份 Word 实验报告可以非常清楚地压缩成：

```
1. Task 设计与迭代
   1.1 Task v1 设计思路
   1.2 Task v2 设计思路
   1.3 Task v1 → Task v2 的改变
        （若无改变，明确写“未修改”并说明原因）

2. Agent 设计与迭代
   2.1 Agent v1 设计思路
   2.2 Agent v2 设计思路
   2.3 Agent v1 → Agent v2 的改变
        （若无改变，明确写“未修改”并说明原因）

3. 总结与反思
   3.1 实验收获
   3.2 可复用的设计经验
```

```
3.3 后续优化方向
```

另外，**Task v2 / Agent v2 不强制必须发生修改**。如果第二阶段与第一阶段完全一致，也是允许的，但必须在实验报告相应位置明确写明“未修改，与 v1 保持一致”，并说明判断依据。

14. DataChallenging 推荐实践流程

完整的执行流程可以概括为：

```
阶段一
──────────────────────────────────

    Design Task
        ↓
    Prepare Data
        ↓
    Write reference.py
        ↓
    Build Agent v1
        ↓
    codesetarena run
        ↓
    Analyze Result
        ↓
    codesetarena build
        ↓
    Task v1 + Agent v1


阶段二
──────────────────────────────────

    Receive Anonymous Tasks
        ↓
    Read Task & Data
        ↓
    Run Agent
        ↓
    Analyze Result
        ↓
Write Improvement Suggestions
        ↓
Improve Task
        ↓
```

```
Improve Agent
        ↓
Task v2 + Agent v2
```

15. 常见问题速查

Q1：Docker 服务没有启动怎么办？

```
systemctl start docker.service
systemctl status docker --no-pager
```

Q2：`docker compose` 不存在怎么办？

先执行：

```
docker compose version
```

如果不存在，再检查：

```
docker-compose version
```

如果旧版 `docker-compose` 可以正常输出，则说明当前安装的是 Compose v1。课程发布包如果要求 Compose v2，应优先按照前文安装 Compose v2。

Q3：`run` 失败怎么办？

如果是阶段一执行run失败优先检查：

```
ls -lah workspace/phase1
python workspace/phase1/reference.py
```

然后确认：

- `task.json` 是否存在；
- 数据文件名是否正确；
- `reference.py` 是否可以独立运行；
- `agent.py` 是否符合课程接口；
- API 配置是否正确；
- 课程服务是否启动。

阶段二则同理

Q6：Agent 能运行，但是答案错误怎么办？

这通常不是“环境安装问题”，而是数据分析逻辑问题。

按顺序检查：

```
1. Task 是否理解正确
2. 字段是否选择正确
3. 数据类型是否正确
4. 过滤条件是否完整
5. 分组 / 聚合是否正确
6. 最终输出类型和格式是否正确
```

Q7：reference.py 和 Agent 的答案不同，应该相信谁？

先检查 `reference.py` 本身是否真正符合 Task 定义。

标准答案程序必须满足：

```
Task 定义正确
      +
Data 支持
      +
reference.py 逻辑正确
      ↓
唯一 Ground Truth
```

如果 reference.py 本身错误，应先修复标准答案程序，而不是为了“通过测试”去让 Agent 模仿错误答案。

---

16. 每次提交前自检

提交前逐项确认：

- [ ] Task 文件完整；
- [ ] 数据文件正确；
- [ ] 问题描述清楚；
- [ ] 答案唯一；
- [ ] `reference.py` 可运行；
- [ ] Agent 符合课程模板；
- [ ] Agent 可运行；
- [ ] Failure Mode 使用课程 FM 编号；
- [ ] 已完成本地测试；
- [ ] 已分析 Agent 成功或失败原因；
- [ ] 提交文件生成成功。

阶段三还需要额外确认：

- [ ] 已根据证据修改 Task v1 → Task v2；
- [ ] 已根据证据修改 Agent v1 → Agent v2；
- [ ] 能说明修改前的问题；
- [ ] 能说明修改后的改善。

---

17. 最小操作版

my_phase这里就是指phase1或者phase2

如果环境已经安装完成，日常完成一个 Task 最常用的操作可以压缩为：

```
# 1. 进入课程目录
cd /path/BigData-v10.1.XX

# 2. 检查课程 CLI
bin/datac --help

# 3. 创建 Task 工作目录
bin/datac init --target /workspace

# 4. 完成以下文件
```

```
# workspace/my_phase/task.json
# workspace/my_phase/data.csv
# workspace/my_phase/reference.py
# workspace/my_phase/agent.py

# 5. 本地运行
bin/datac run --workspace /workspace

# 6. 生成提交文件
 bin/datac submission build \
  --input /workspace \
  --student-id 4725200597 \
  --phase P1
```

---

18. DataChallenging 一句话理解

```
设计 Task
   ↓
准备 Data + Ground Truth
   ↓
构建 Agent
   ↓
真实运行
   ↓
发现成功 / 失败
   ↓
分析原因
   ↓
匿名 Task 交叉分析
   ↓
Task v2 + Agent v2
```

附页A｜正式评测：精选测试集 + 攻防对抗

> **两种模式同时计入最终成绩**：精选测试集考察 Agent 的综合解题能力；攻防对抗考察 Agent 进攻表现与 Task 防守能力。

① 提交什么：最终参评成果

成果	正式评测使用规则
Agent v2（矛）	**唯一**参加最终智能体评测；Agent v1 仅用于过程测试与迭代分析。
Task v1 + Task v2（盾）	两道题**分别审核**，合格的题目都有机会参与评测，最多 2 道。

② 什么有资格参评：规范与质量审核

Task 须题意明确、没有歧义、参考程序与标准答案正确，且学生自己的 Agent 能成功求解；应针对课程规定的 Failure Mode 设计。Agent 也须通过运行与提交规范审核。**超时、资源超限和运行时错误不能作为预期 Failure Mode 的有效证明。** 未通过审核的成果不具备对应参评资格，因此每名学生实际可能是 **0～1 个矛、0～2 个盾**。

③ 怎么评测：两种模式同时进行

模式一：精选测试集评测	模式二：攻防对抗评测（Elo Rating）
从有效 Task 中按失败模式覆盖、原创性、区分度等筛选优质题目；全部合格 **Agent v2** 统一解题。	**Agent v2 为矛**，挑战其他同学的有效 Task；**Task v1/v2 为盾**，接受其他 Agent 挑战。
Agent 解对的精选题越多，表现越好；**题目入选精选测试集可获得额外加分**。	使用 **Elo Rating** 分别形成进攻与防守排名，并据此计分。

④ 如何计分：两种模式均影响最终成绩

精选测试集成绩体现 Agent 解题能力及入选任务贡献；攻防对抗成绩体现 Agent 进攻与 Task 防守排名。**具体成绩权重、Elo 配对数量、排名换分规则以后续正式评分细则为准**，此处不预设数值。

必须记住的 4 条规则：① 两轮 Task 分别审核、都可能参评；② 最终只评 Agent v2；③ Task 必须自己 Agent 能解；④ 程序崩溃、超时及资源超限不算规定失败模式的有效证明。

模型说明：正式评测计划采用 DeepSeek Flash（暂定 `deepseek-v4.1-flash`）及可能的其他未公开基座模型。Agent 应具备跨模型适应能力，不能针对单一模型结果硬编码。

附页B｜最终成绩：由 4 项评测结果综合计算

> **成绩同时看出题质量与解题能力**：题目质量、精选测试集解题表现、Elo 进攻排名、Elo 防守排名四项结果共同决定最终成绩。

成绩组成	评什么	如何得到分数
① 题目质量得分（(S_Q)）	Task v1 / Task v2 的有效性、原创性与挑战性	通过任务质量审核；高质量题目入选精选测试集可获得额外加分。

成绩组成	评什么	如何得到分数
② 测试集解题得分（(S_T)）	Agent v2 对精选测试集的综合解题能力	Agent v2 统一解题，正确解决的有效题目越多，表现越好。
③ Elo 进攻得分（(S_A)）	Agent v2 作为“矛”，挑战其他同学的有效 Task	根据攻防结果计算 Elo 进攻排名，再按公布的规则转换成分数。
④ Elo 防守得分（(S_D)）	Task v1 / v2 作为“盾”，接受其他同学 Agent 的挑战	根据攻防结果计算 Elo 防守排名，再按公布的规则转换成分数。

最终成绩计算关系（建议表达，非已确定公式）

\最终成绩计算关系

同时保留了加权成绩关系：

$$S_{\text{final}} = w_Q S_Q + w_T S_T + w_A S_A + w_D S_D$$

同时保留了加权成绩关系：**W为对应权重。**

S_Q为题目质量得分，S_T为精选测试集解题得分，S_T为精选测试集解题得分，S_A为Elo进攻得分，S_D为Elo防守得分。

特别说明： 文档明确将此公式标记为建议表达，而非已经确定的正式评分公式。四项权重、精选题入选加分和 Elo 排名换分规则均留待课程组正式公布。

重要说明：这是用于说明成绩构成的**建议加权表达式**，不是已经公布的正式计分公式。四项权重、题目入选加分标准、无效成果处理方式、Elo 对抗数量和排名换分方法，均以课程组后续发布的正式评分细则为准。

课堂强调：① Task v1/v2 与 Agent v2 评价对象不同；② Elo 进攻与 Elo 防守是两套独立排名；③ 只提高 Agent 答题正确率，不等于一定获得最高总成绩。